Thoughts on the exams?

clicker.cs.illinois.edu

Q1

~Code~
340

CS 340

Python



Updates

1. MP5 Wallet due tonight

1. MP6 out today

higher 80% - lower 20%

1. Exam 2
   a. Retake on Thursday
   b. Submit regrades via Prairie Learn “report an error” on question.

Python

Today's LGs - Be able to read and write python code

Be able to complete MP6 in python

What is your experience with Python?

clicker.cs.illinois.edu

Q2

~Code~
340

62%
written basic code

21%
written complex code

C to Python Example

```c
#include <stdio.h>
#include <stdlib.h>

int main(){
    int x = 4;
    int* y = malloc(12);
    y[0] = 9; y[1] = 2; y[2] = 5;
    y = realloc(y, 16);
    y[3] = x;
    for(int i = 0; i < 4; i++){
        if(y[i] == 4){
            printf("yay");
            return 0;
        }
    }
    printf("no");
    return 0;
}
```

x = 4

y = [9, 2, 5]

y.append(x)

```
if 4 in y:
    print("yay")
print("no")
```

A few favorites about Python

```
1  print('hello')
2  x = {'abc':10, 'bye':6}
3  print(x)
4
5  y = [1, 4, 90]
6  if 10 in y:
7      print('hi')
8
```

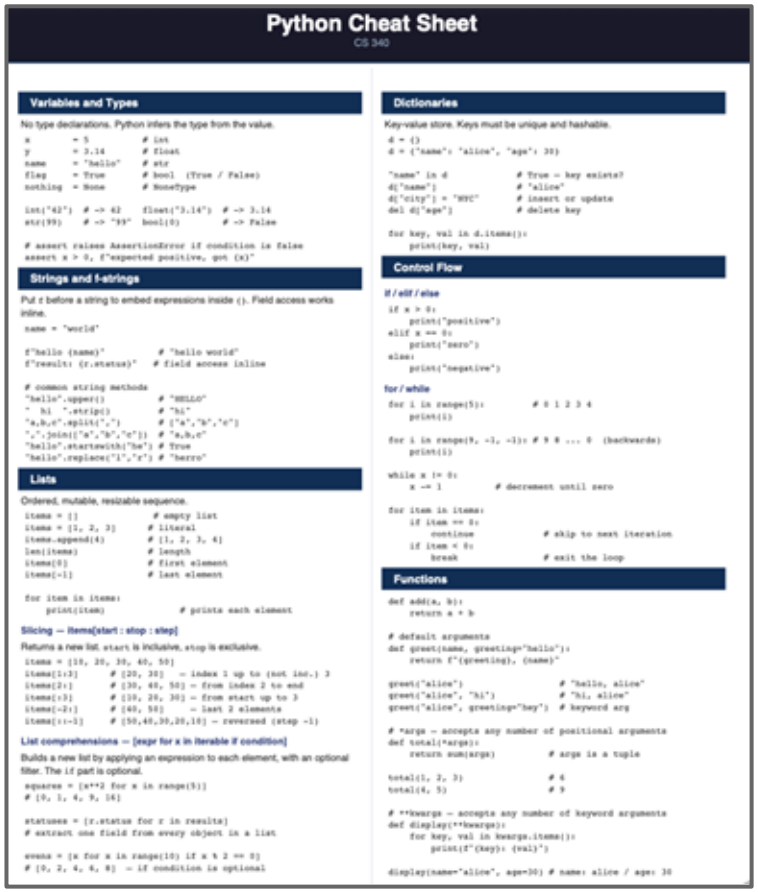
no main

PROBLEMS OUTPUT TERMINAL 2

TERMINAL bash

```
root@0e9b25a8fb50:/workspaces/mp7-testing# python3 test_class.py
hello
{'abc': 10, 'bye': 6}
```

Python Cheat Sheet



Python Cheat Sheet

CS 340

Variables and Types

No type declarations. Python infers the type from the value.

```
x       = 5           # int
y       = 3.14        # float
name    = "hello"     # str
flag    = True        # bool  (True / False)
nothing = None        # NoneType

int("42")  # -> 42    float("3.14")  # -> 3.14
str(99)    # -> "99"  bool(0)        # -> False

# assert raises AssertionError if condition is false
assert x > 0, f"expected positive, got {x}"
```

Strings and f-strings

Put `f` before a string to embed expressions inside `{}`. Field access works inline.

```
name = "world"

f"hello {name}"          # "hello world"
f"result: {r.status}"    # field access inline

# common string methods
"hello".upper()          # "HELLO"
"  hi  ".strip()         # "hi"
"a,b,c".split(",")       # ["a","b","c"]
",".join(["a","b","c"])  # "a,b,c"
"hello".startswith("he") # True
"hello".replace("l","r") # "herro"
```

Lists

Ordered, mutable, resizable sequence.

```
items = []               # empty list
items = [1, 2, 3]        # literal
items.append(4)          # [1, 2, 3, 4]
len(items)               # length
items[0]                 # first element
items[-1]                # last element

for item in items:
    print(item)              # prints each element
```

Slicing — `items[start : stop : step]`

Returns a new list. `start` is inclusive, `stop` is exclusive.

```
items = [10, 20, 30, 40, 50]
items[1:3]      # [20, 30]   — index 1 up to (not inc.) 3
items[2:]       # [30, 40, 50] — from index 2 to end
items[:3]       # [10, 20, 30] — from start up to 3
items[-2:]      # [40, 50]     — last 2 elements
items[::-1]     # [50,40,30,20,10] — reversed (step -1)
```

List comprehensions — `[expr for x in iterable if condition]`

Builds a new list by applying an expression to each element, with an optional filter. The `if` part is optional.

```
squares = [x**2 for x in range(5)]
# [0, 1, 4, 9, 16]

statuses = [r.status for r in results]
# extract one field from every object in a list

evens = [x for x in range(10) if x % 2 == 0]
# [0, 2, 4, 6, 8]  — if condition is optional
```

Dictionaries

Key-value store. Keys must be unique and hashable.

```
d = {}
d = {"name": "alice", "age": 30}

"name" in d              # True — key exists?
d["name"]                # "alice"
d["city"] = "NYC"        # insert or update
del d["age"]             # delete key

for key, val in d.items():
    print(key, val)
```

Control Flow

`if / elif / else`

```
if x > 0:
    print("positive")
elif x == 0:
    print("zero")
else:
    print("negative")
```

`for / while`

```
for i in range(5):         # 0 1 2 3 4
    print(i)

for i in range(9, -1, -1): # 9 8 ... 0  (backwards)
    print(i)

while x != 0:
    x -= 1           # decrement until zero

for item in items:
    if item == 0:
        continue             # skip to next iteration
    if item < 0:
        break                # exit the loop
```

Functions

```
def add(a, b):
    return a + b

# default arguments
def greet(name, greeting="hello"):
    return f"{greeting}, {name}"

greet("alice")                  # "hello, alice"
greet("alice", "hi")            # "hi, alice"
greet("alice", greeting="hey")  # keyword arg

# *args — accepts any number of positional arguments
def total(*args):
    return sum(args)          # args is a tuple

total(1, 2, 3)                # 6
total(4, 5)                   # 9

# **kwargs — accepts any number of keyword arguments
def display(**kwargs):
    for key, val in kwargs.items():
        print(f"{key}: {val}")

display(name="alice", age=30) # name: alice / age: 30
```

website content this lecture python cheat sheet

Variables, Types, Dictionaries, Lists

```
x = 5   # int
y = 3.14  # float
name = "hello"  # string
flag = True  # bool
int("42") ==> 42
str(42) --> "42"
```

```
d = {}
d = {"name": "alice", "age": 30}
if "name" in d:   ← checks for key
for key, val in d.items():
    print(key, val);
ls = []
ls.append(5)
```

List Slicing

[start : stop : step]

- start: inclusive, defaults to 0
- stop: exclusive, defaults to len(lst)
- step: defaults to 1

lst = [1, 2, 3, 4, 5]

sub_1 = lst[1:3:1] = [2, 3]

sub_2 = lst[2::2] = [3, 5]

lst[-1] → 5

What prints for the last one?

```
3  itms = [10, 20, 30, 40, 50]
4
5  print(itms[1:3])
6  print(itms[2:])
7  print(itms[:3])
8  print(itms[-3:2])
```

clicker.cs.illinois.edu

Q3

~Code~
340

List Comprehension

Builds a new list by applying an expression to each element with an optional filter.

```
lst = [1, 2, 3, 4]
sq = [x**2 for x in lst if x%2==0] => [4, 16]
sq = [x**2 for x in lst] => [1, 4, 9, 16]
```

Range

range(start, stop, step) => returns a sequence of values

start → 0
stop → 5
step → 1

for i in range(5), => 0, 1, 2, 3, 4

1, 2, 3, 4

sq = [x**2 for x in range(1, 5)]

=> [1, 4, 9, 16]

What prints?

```
2 res = [i+2 for i in range(0,8,2) if i > 4]
3 print(res)
```

[8]

[0, 2, 4, 6]

clicker.cs.illinois.edu

Q4

~Code~
340

Challenge: Does this code pass the assert?

```
1  def func(nums, k):
2      result = []
3      for i in range(len(nums) - k + 1):
4          window = nums[i:i+k]
5          result.append(max(window))
6      return result
7
8  assert func([1, 3, 2, 5, 4], 3) == [3, 5, 4]
```

clicker.cs.illinois.edu

Q5

~Code~
340



X

0, 1, 2

5 - 3 + 1 = 3

trigger if statement is not true

① window = [1, 3, 2] = 3
result = [3]

② window [3, 2, 5]
result = [3, 5]

window

③ [2, 5, 4]
result = [3, 5, 5]

Python Vs. C

- compiles to machine code
- machine runs on the cpu
  - very fast

-

- loaded into a program called the python interpreter
  - interpret your python code
  - keep track of things

Python Vs. C Analogy Attempt



robot that only speaks german

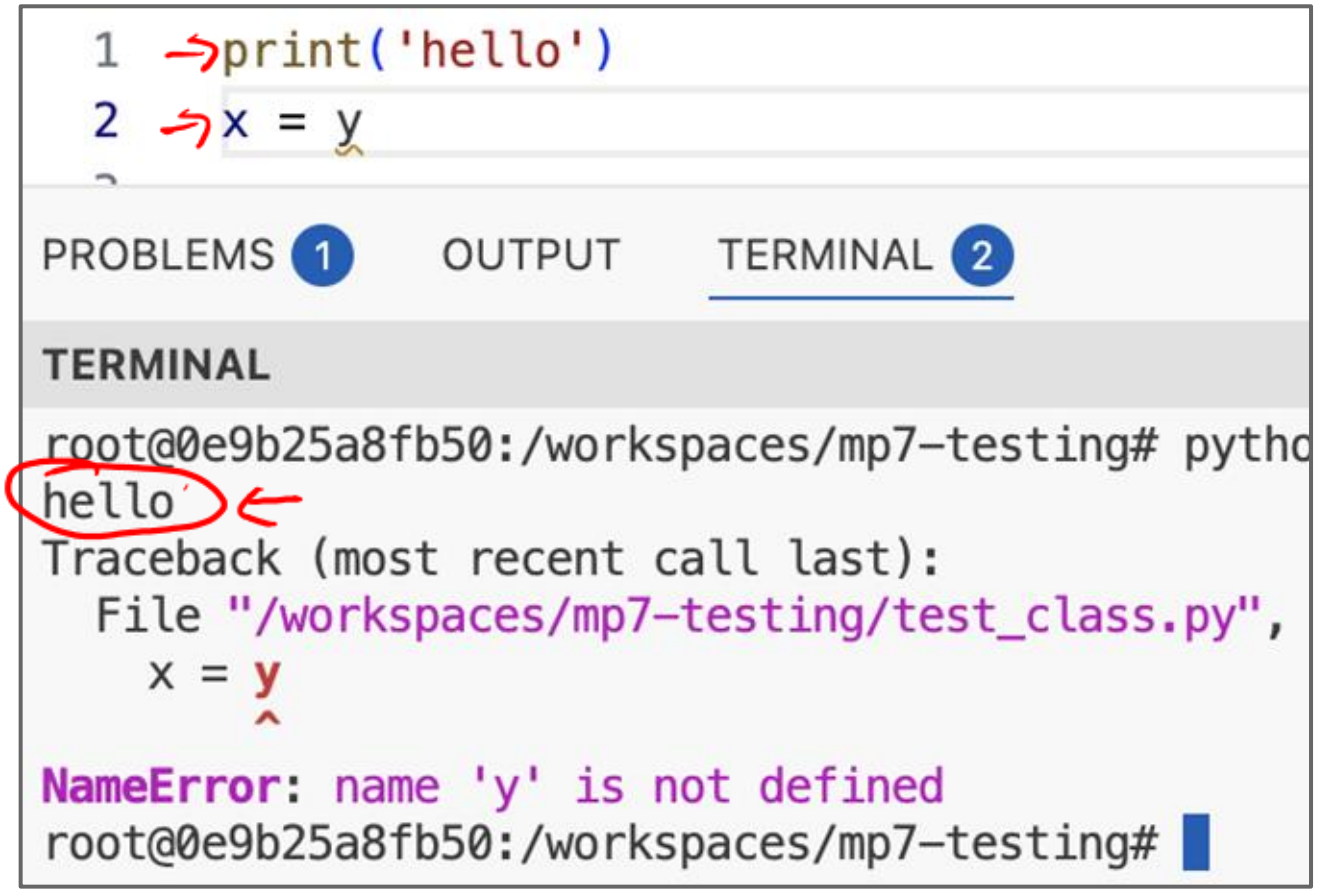
C -> clunky english that translates to german

"Jule is a professor" -> Jule ist eine professorine.

Python -> slang english -> "she dabes as a prof"

German instructions

Errors


1 print('hello')
2 x = y
PROBLEMS 1
OUTPUT
TERMINAL 2
TERMINAL
root@0e9b25a8fb50:/workspaces/mp7-testing# pytho
hello
Traceback (most recent call last):
File "/workspaces/mp7-testing/test_class.py",
x = y
NameError: name 'y' is not defined
root@0e9b25a8fb50:/workspaces/mp7-testing#

Performance Favors Library Code

```
1  total = 0
2  for x in range(10000000):
3      total += x
4  print(total)
```

→ slow

C code

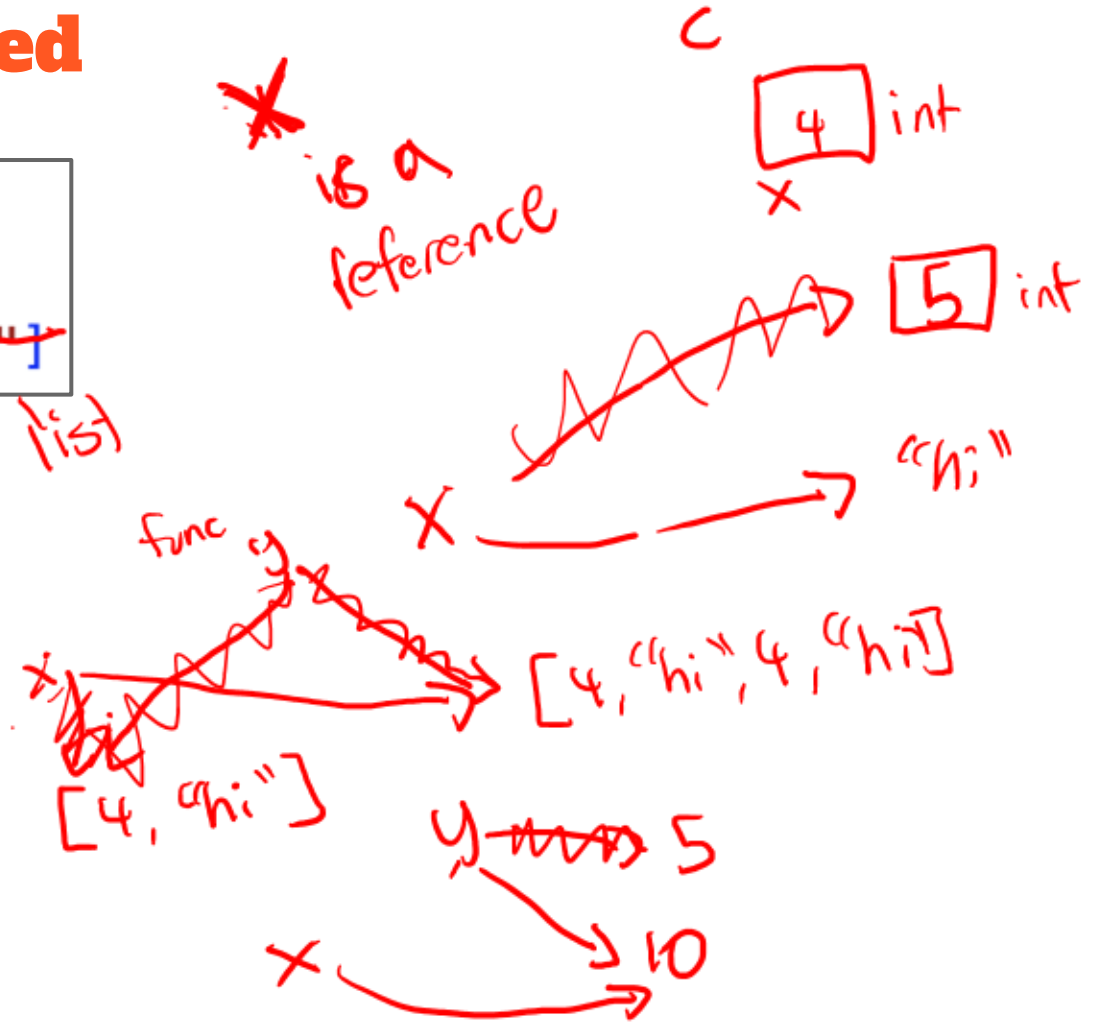
```
1  total = sum(range(10000000))
2  print(total)
```

→ fast

Dynamically Typed

```
1    x = 'hi'
2    x = 5
3    x = [4, 10, "hi"]
```

```
1    def func(y):
2        y = y+y
3        return y
4
5    x = [4, "hi"]
6    x = func(x)
7    x = func(5)
```



Tricky Examples

- in place
- a copy

```
1    x = 5
2    y = x
3    y += 5
4    print(x)
```

copy

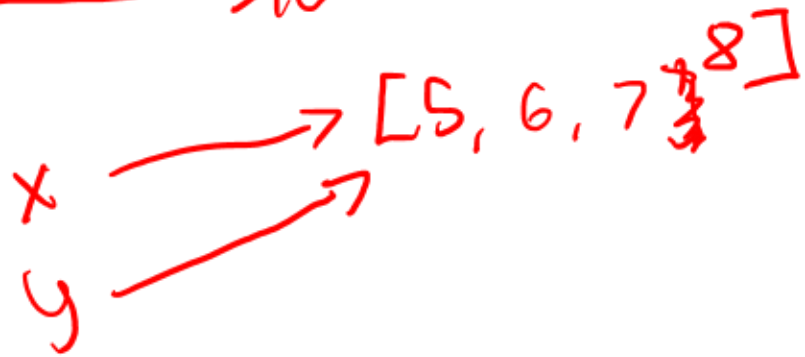
5




```
1    x = [5, 6, 7]
2    y = x
3    y.append(8)
4    print(x)
```

in place

[5, 6, 7, 8]



What prints?

```
1    z = [1, 2]
2    y = [1, 2]
3    y.append(6)
4    print(z)
```

[1, 2]

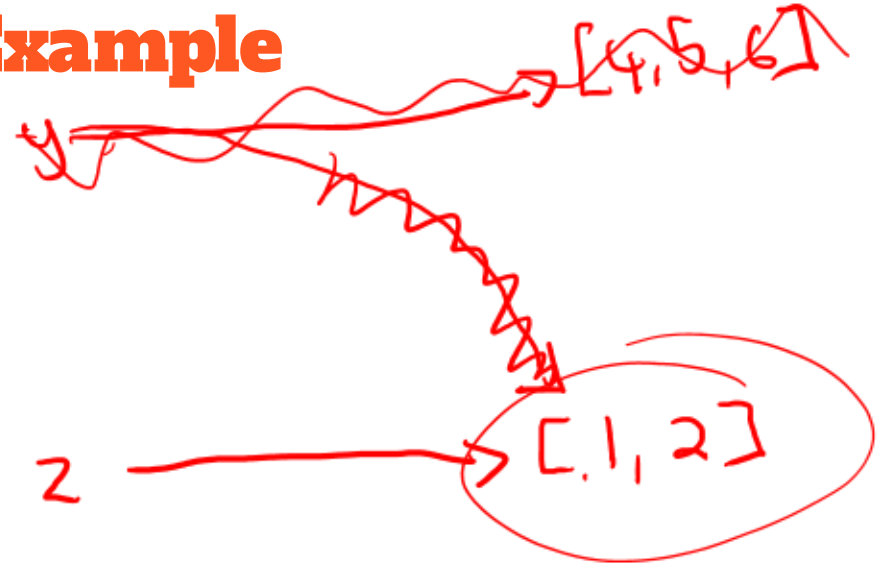
clicker.cs.illinois.edu

Q6

~Code~
340

Tricky Function Example

```
1    def func(y):
2        y = [4, 5]
3        y.append(6)
4
5    z = [1, 2]
6    func(z)
7    print(z)
```



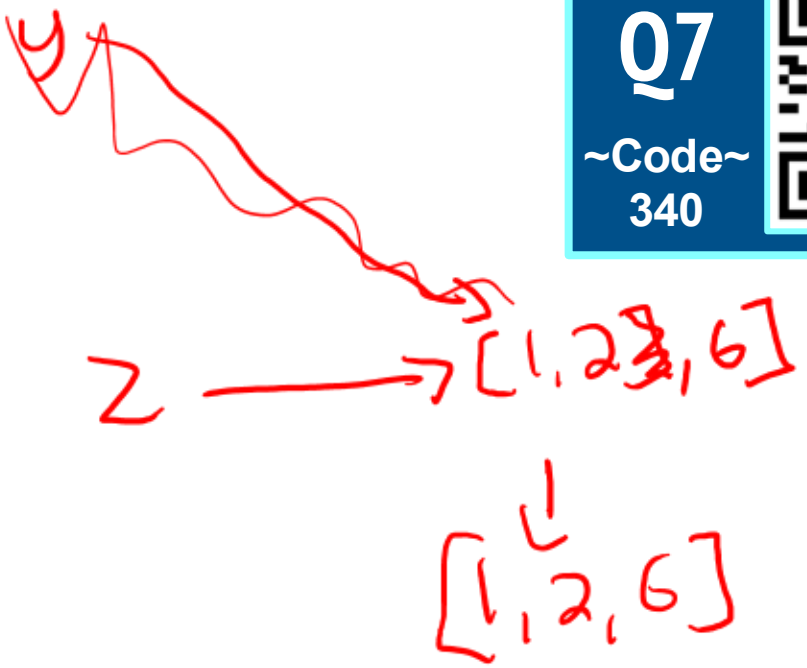
What prints?

```
1    def func(y):
2
3        y.append(6)
4
5    z = [1, 2]
6    func(z)
7    print(z)
```

clicker.cs.illinois.edu

Q7

~Code~
340



Challenge: What prints?

clicker.cs.illinois.edu

Q8

~Code~
340

```
1    def func(y):
2        sorted(y)
3
4    z = [1, 5, 2]
5    func(z)
6    print(z)
```

makes a copy
not in-place

1, 5, 2

Optional Types for Readability

```
1    def func(y):
2        y += y
3        return y
4
5    x = 5
6    x = func("x")
7    print(x)
```

```
1    def func(y: int) -> int:
2        y += y
3        return y
4
5    x: int = 5
6    x = func(x)
7    print(x)
```

Would this run successfully?

```
1  def total_price(items, tax):
2      total = 0.0
3      for item in items:
4          total += item[1]
5      return total * (1 + tax)
6
7  lst = [20.1, 34.45, 4, 35.56]
8  total_price(lst, 0.1)
```

clicker.cs.illinois.edu

Q9

~Code~
340

Would this run successfully?

```
1  def total_price(items: list[tuple[str, float]], tax: float) -> float:
2      total = 0.0
3      for item in items:
4          total += item[1]
5      return total * (1 + tax)
6
7  lst = [0.1, 34.45, 4, 35.56]
8  total_price(lst, 0.1)
```

[("drink", 6)]

clicker.cs.illinois.edu

Q10

~Code~
340

Scope

```
2  x = 4
3  if True:
4      x = 1
5
6  print(x)
```

= 5

only by function or file

↑ module

```
if True
    x = 6
print(x)
```

✓

Function Scope

```
1  x = 4
2  def func():
3      global x
4      x += 1
5
6  func()
```

```
1  x = 4
2  def func():
3      x += 1
4
5  func()
```

Ticky Example: What happens?

```
1 x = 4
2 def func():
3     global x
4     x += 1
5 func()
6 print(x)
```

4

clicker.cs.illinois.edu

Q11

~Code~
340

What happens?

```
1  x = 4
2  def func():
3      global x
4      x += 1
5
6  func()
7  func()
8  func()
9  print(x)
```

clicker.cs.illinois.edu

Q12

~Code~
340

What happens?

```
1  x = 4
2  def func():
3      x = 5
4      x += 1
5  func()
6  func()
7  print(x)
```

4

clicker.cs.illinois.edu

Q13

~Code~
340

What happens?

```
2  x = 5
3  for i in range(0, 3):
4      x += 2
5  print(i)
```

0, 1, 2

2

slope? → function file

range? → start, end

clicker.cs.illinois.edu

Q14

~Code~ 340

Python Sequences

```python
def func():
    return 10, "hi", {3: "hi", 4:"bye"}

a, b, c = func()
print(c)
```

PROBLEMS OUTPUT TERMINAL 2

TERMINAL

```
root@0e9b25a8fb50:/workspaces/mp7-testing# python3 test_class.py
{3: 'hi', 4: 'bye'}
```

MP6 - Scheduler Tests

given

- spec of what the scheduler should do
- working version of the scheduler

---

autograder

- 23 buggy versions

MP6 - Scheduler Tests

- using the worksheet

get a queue of jobs

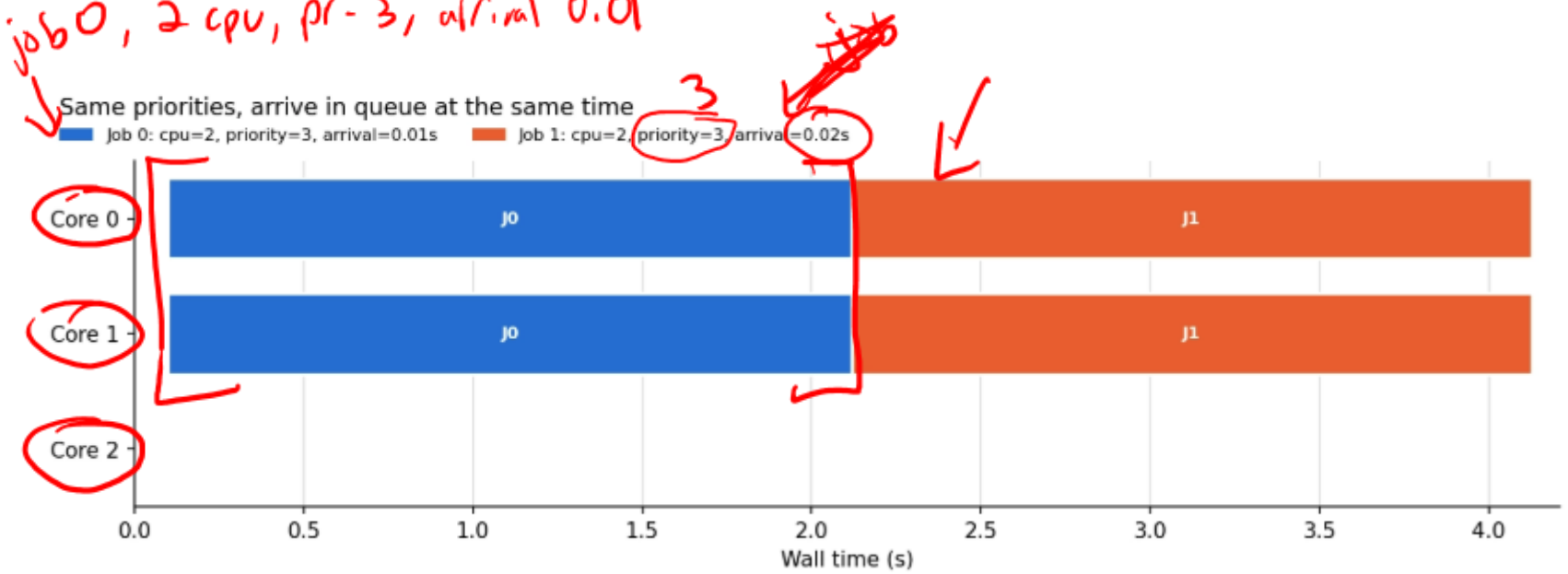
every 0.1 seconds

- cpu core req.
- workload
- priority (0 ↙ low – 10 ↙ high)
- arrival time
- job id

priority → arrival time → job id

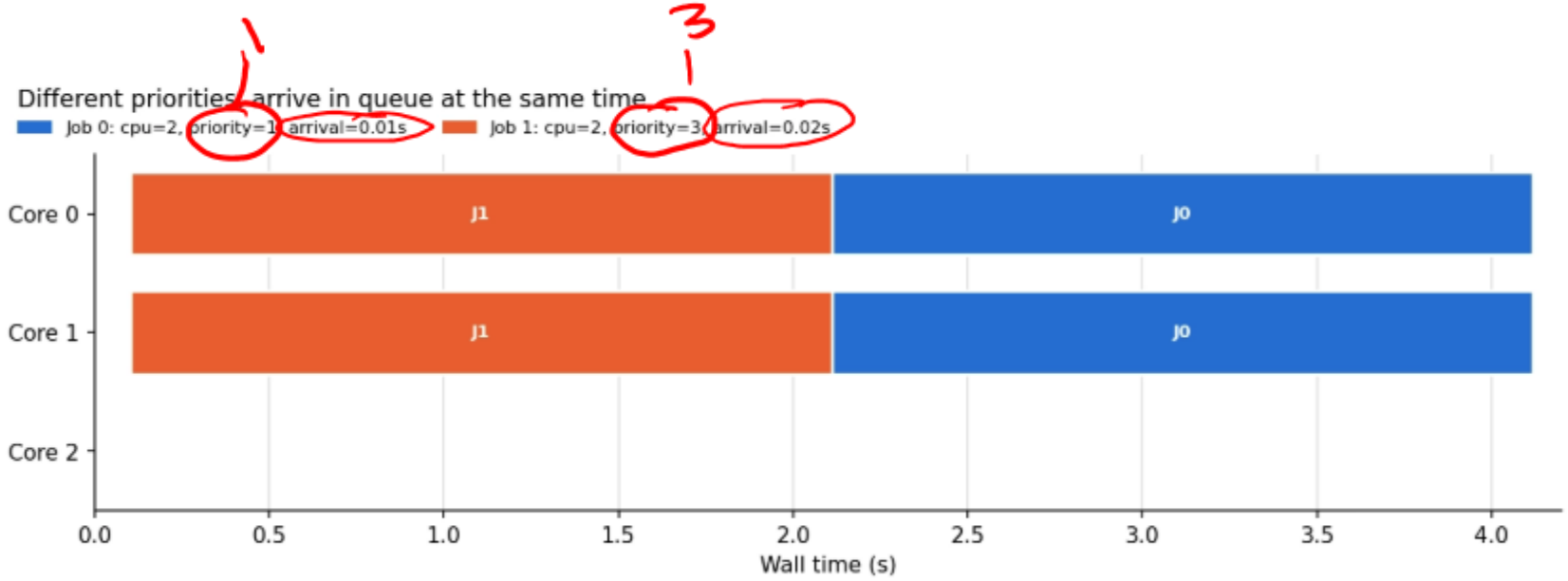
MP6 - Scheduler Tests

Example 1 - Same priorities different arrival times



MP6 - Scheduler Tests

Example 2 - Different priorities no preemption



MP6 - Scheduler Tests

Example 3 - Different priorities and preemption

